→ Movie : "Ek Language ka Janm"

→ Scene 1 : Jungle of characters

Screen par likha aata hai :

3 + 4 * 5

computer dekhta hai :

'3' ' ' '+' ' ' '4' ' ' '*' ' ' '5' → characters

computer ko kuchh samajh nahi aa raha hai, yaha entry hoti hai pahle hero ki :

* Hero 1 : The Scanner

Scanner : mai bekar characters ko meaning dunga

(wo dheere dheere chalna shuru karta hai, aur dekhta hai)

• '3'

Scanner : Are, ye to NUMBER hai

'3' → NUMBER

• ' '

Scanner : Baba, whitespace, mere ko kuchh dikhayi nahi de raha

director : koi bat nahi, ek kadam aage badh ja.

• '+'

Scanner : Are, ye to PLUS hai

'+' → PLUS

director : haa, ab aise hi pura jungle ghum lo.

PRAGATI
Page:
Date:

(Ab jungle nahi raha, sara ghum liya)

Ab hamare paas hai:

[~~NUMBER(3), PLUS, NUMBER(4), STOR, NUMBER(5),~~ EOF]

(Ab characters ka jungle → tokens ki list ban gayi)

~~Scene~~ director: Scanner ab tumhara kam khatam, lights dim kar do.

→ Scene 2: Flat World

(Ab stage par tokens pade hai)

(Sab ek line me khade hai, par problem hai sab flat hain, isiliye pahle karna hai nahi pata:)

yaha, entry hoti hai dusre hero ki:

* Hero 2: The Parser

parser: main in sabko arrange karunga

(parser ke paas grammar naam ka map hai) vs map me likha hai:

expression → term

term → factor (("+" | "-") factor)*

factor → unary (("*" | "/") unary)*

unary → "-" unary | primary

primary → NUMBER | "(" expression ")"

Passer: mai sabse upar se shuru karunga → expression se.

स्पष्ट मूल्य-उचित मूल्य"

→ Scene 3: The Journey Begins

(Parser khada hai first token par)

Parser: Expression parse karo

Expression: main term hoon.

term: main factor hoon.

factor: main unary hoon.

unary: main primary hoon.

(primary dekhta hai, NUMBER(3))

primary: Theek hai, ye ek literal hai

(Tree ka pahla leaf ban gaya)

→ Scene 4: The Conflict

(Parser upar chadh kar dekhta hai)

Parser: Agla token kya hai?

token: PLUS

(term ke grammar me likha hai, jab tak PLUS mile, naya branch banao)

Parser: Haha, PLUS ko consume kar leta hun. aur right side parse karunga.

→ Scene 5: Deep Nesting

(Right side parse karte waqt parser dekhta hai)

→ 4 * 5

(OB factor rule kahta hai:

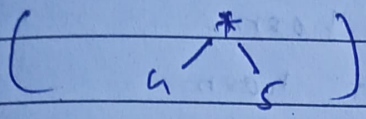
("*" unary)*)

Parser: pahle 4 ko literal banata hoon,

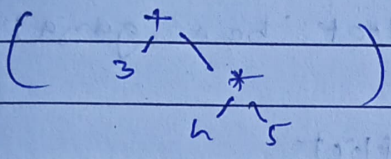
Parser: Are, STAR bhi hai

PRAGATI
Page:
Date:

STAR : mujhe pahle solve karo
parser : okay, ek naya subtree bana
leta hoon.



→ Scene 6 :— Final Tree
(Ab parser wapas plus par aata hai,
Aur final tree ban jata hai)



director : parser, tumhara kam khatam

→ Scene 7 : The Enterpreter Enters
(Ab teesra hero, aata hai — interpreter)
(Tree ko dekhte hue)
interpreter; Ab main is structure ko
execute karunga
(wo sabse neeche jata hai)
interpreter : 4*5 = 20
(phir upar aata hai)
interpreter; 3+20 = 23
(result mil gya)

→ Scene 8 : Parser learns to walk
(Ab parser ko tokens, ke beech
chalna seekhna hai, uske paas ek
list hai: [NUMBER(3), PLUS, ...]

(Parser ke paas ek pointer hai)
pointer: main abhi is token ke paas khada hoon.
parser: sare tools, start ho jao.

Tool 1 — peek()
passer: main dekhna chahta hu, ki mere saamne kaun khada hai, bina aage badhe.
peek(): aapke saamne NUMBER(3) khada hai.

Tool 2 — advance()
parser: advance(), is token ko accept karo, aur main ek step aage badhta hoon.
(ye karta hai: current token return and current++)

Tool 3 — match()
parser: Agar mere saamne PLUS ya MINUS ho, to use consume karo.
(ye grammar implement karne ka tool hai)

Tool 4 — check()
parser: Bas confirm karo. ki current token PLUS hai ya nahi, consume mat karo.

Tool 5 — isAtEnd()
parser: kya main EOF par pahuch gya hoon.